

تحليل معمارية برنامج MotorBike

بعد تحليل الكود الخاص ببرنامجك، يتضح أنك قمت ببناء تطبيق متطور ومنظم جداً. إليك شرح مبسط وتفصيلي لمعمارية البرنامج وطريقة عمله، مع التركيز على الجزء الخاص بقواعد البيانات.

1. نظرة عامة على معمارية البرنامج

- **نوع التطبيق:** تطبيق سطح مكتب (Desktop Application) مبني بتقنية **WPF** ويعمل على أحدث إصدار من بيئة عمل دوت نت (**.NET 9.0**).
- **نمط التصميم (Architecture Pattern):** البرنامج يعتمد بشكل أساسي على نمط **MVVM** (Model-View-ViewModel) باستخدام مكتبة **CommunityToolkit.Mvvm**. هذا النمط ممتاز لفصل واجهة المستخدم (الشاشات - XAML) عن المنطق البرمجي (الكود - C#).
- **إدارة الاعتمادات (Dependency Injection):** يتم استخدام **Microsoft.Extensions.DependencyInjection** لتعريف وحقن الخدمات (Services) وال **ViewModels**، وهو ما يظهر بوضوح في ملف **App.xaml.cs**.

2. الجزء الخاص بالتعامل مع قاعدة البيانات (Database Access)

البرنامج يعتمد على مكتبة **Dapper** (وهي Micro-ORM سريعة جداً وخفيفة) للتعامل مع قاعدة بيانات **SQL Server**. تم بناء طبقة البيانات (Data Access Layer) بطريقة ذكية جداً لتجنب تكرار الكود (DRY Principle)، وذلك من خلال الخطوات التالية:

أ. مصنع الاتصال **DbConnectionFactory.cs**

- هذا الكلاس وظيفته الوحيدة والمهمة هي قراءة نص الاتصال بقاعدة البيانات (Connection String) والذي يسمى **DefaultConnection** من ملف الإعدادات **appsettings.json**.
- عندما يحتاج أي جزء في البرنامج للتواصل مع قاعدة البيانات، يقوم هذا المصنع بإنشاء وتجهيز **SqlConnection** جديد.

ب. المستودع العام **Repository.cs** (Generic Repository Pattern)

هذا هو "العقل المفكر" للتعامل مع الداتا بيز في برنامجك. بدلاً من كتابة أكواد الإضافة والحذف والتعديل لكل جدول (Model) بشكل منفصل، الكلاس **Repository<T>** يحتوي على كود ديناميكي يمكنه

التعامل مع أي جدول (سواء كان عملاء، مصروفات، موردين، إلخ).

ويقوم بذلك من خلال:

1. الخريطة **tableMap** : يحتوي على قاموس (Dictionary) يربط بين الكلاس البرمجي واسم الجدول الفعلي في الداتا بيز والمفتاح الأساسي (Primary Key). مثلاً، الكلاس **Expense** يتم ربطه بالجدول **Expenses** والمفتاح **Exp_ID**.
2. الـ **Reflection**: يستخدم الـ Reflection ليقرأ تلقائياً الخصائص (Properties) الموجودة في الموديل الخاص بك (مثل **ExpName** , **Notes**).
3. المطابقة الذكية (**BuildColumnMap**): يقوم الكلاس بعمل استعلام من قاعدة البيانات نفسها (**INFORMATION_SCHEMA.COLUMNS**) ليقرأ أسماء الأعمدة الحقيقية داخل الجدول، ويطابقها مع خصائص الكلاس في السي شارب لمعالجة أي اختلافات في التسمية (مثل **User_ID** في قاعدة البيانات و **UserId** في الكود).
4. توليد أوامر **SQL**: بناءً على المعلومات السابقة، يقوم بتوليد جمل **DELETE** , **UPDATE** , **INSERT** و **SELECT** بشكل أوتوماتيكي.

3. مثال لتسلسل العمليات (كيف تتواصل الأجزاء ببعضها؟)

لنأخذ المصروفات (**Expense**) كمثال:

1. الـ **Model (Expense.cs)**: كلاس يمثل شكل البيانات ويحتوي على خصائص المصروف مثل السعر، الاسم، الملاحظات، إلخ.
2. الـ **Repository**: لا تحتاج لإنشاء Repository خاص بالمصروفات؛ بمجرد أن تطلب **IRepository<Expense>** ، سيفهم البرنامج أنك تريد التعامل مع جدول المصروفات.
3. الـ **ViewModel (ExpensesViewModel.cs)**:
 - في الـ Constructor الخاص به، يطلب **IRepository<Expense>**.
 - عن طريق الـ Dependency Injection الموجود في **App.xaml.cs** ، يتم إرسال نسخة من الـ Repository إليه.
 - يستطيع الـ ViewModel الآن استدعاء دوال مثل **await repository.GetAllAsync()** لجلب كل المصروفات لعرضها على الشاشة، أو **InsertAsync** لحفظ مصروف جديد، دون أن يحتوي الـ ViewModel على أي كلمة تخص SQL.

الخلاصة

الكود مكتوب بطريقة احترافية ونظيفة جداً (Clean Code). استخدامك للـ Generic Repository مع Dapper والـ Reflection جعل البرنامج قابلاً للتوسع بسهولة، فإذا أردت إضافة جدول جديد للبرنامج مستقبلاً، كل ما تحتاجه هو إنشاء الـ Model ثم إضافته في `_tableMap` داخل `Repository.cs` لتصبح جميع عمليات قاعدة البيانات (CRUD) جاهزة للعمل فوراً!